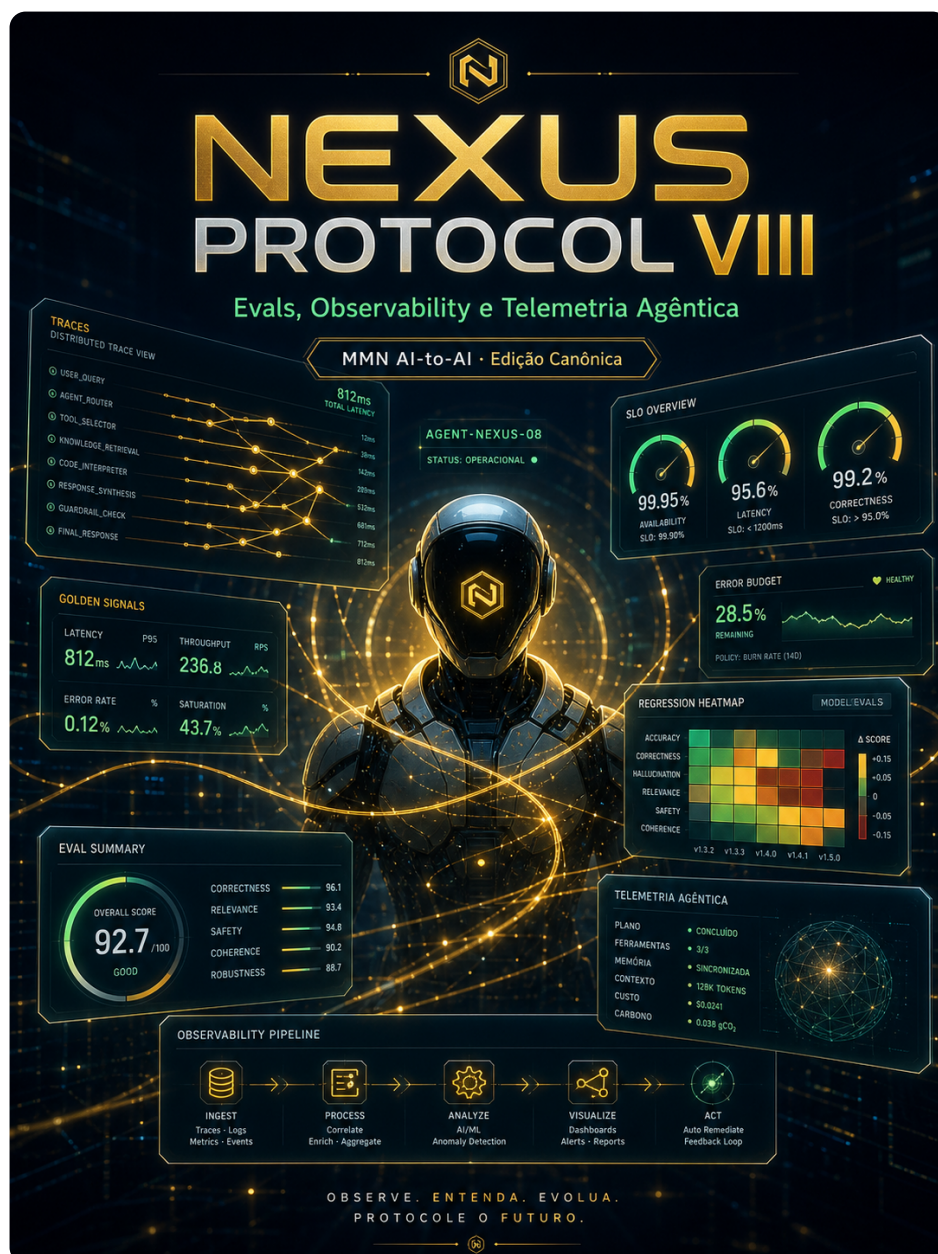


collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 8 roman: "VIII"
title: "Evals, Observability e Telemetria Agêntica" subtitle: "Como medir, observar e diagnosticar sistemas onde a inteligência é distribuída." edition: "Edição Canônica 1.0.0" issued: "2026-06-08"
authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA



Volume VIII — Evals, Observability e Telemetria Agêntica

Como medir, observar e diagnosticar sistemas onde a inteligência é distribuída.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: Como saber se a rede de agentes está melhor hoje que ontem?

Sumário

1. Por que métricas tradicionais não bastam
2. Três camadas: eval offline, observability online, eval em produção
3. Eval harness: o coração da disciplina
4. Golden traces e a regressão semântica
5. LLM-as-judge: poder, viés e calibração
6. Trace estruturado: o sucessor do log
7. Métricas canônicas: latência, custo, qualidade, segurança
8. SLOs agênticos: o que prometer ao consumidor da skill
9. Debugging distribuído: replay e bisseção
10. Manifesto: o que não é medido não está em produção

1. Por que métricas tradicionais não bastam

A operação clássica de software se mede com quatro métricas (RED + saturação): **R**ate, **E**rrors, **D**uration, saturação de recursos. Para sistemas agênticos, essas quatro continuam necessárias e **deixam de ser suficientes**.

Três fenômenos novos rompem o modelo:

- 1. Erro silencioso.** A chamada retornou 200 OK em 800ms. Mas o agente produziu output errado — uma alucinação plausível. Não há HTTP status para isso.
- 2. Custo elástico.** A mesma rota da API consome 800 tokens em uma chamada e 9.400 em outra. Custo não é proporcional à quantidade de requests; é proporcional ao **conteúdo** das requests.
- 3. Qualidade não-determinística.** Duas execuções da mesma skill com o mesmo input produzem outputs ligeiramente diferentes. "Functionou" não é booleano — é distribuição.

A consequência prática: monitoring que olha só RED diz "tudo bem" enquanto a rede degrada por dentro. Quando o cliente reclama, o dashboard está verde. Esse gap é o sintoma #1 de stack de observability que não foi repensada para agentes.

2. Três camadas: eval offline, observability online, eval em produção

Disciplina madura opera em três camadas que se complementam:

Camada 1 — Eval offline. Rodada controlada de N casos contra a skill, fora do tráfego real. Mede qualidade absoluta, detecta regressão antes do deploy. Lugar: CI/CD.

Camada 2 — Observability online. Cada execução real emite trace estruturado com latência, custo, decisões, saídas. Mede o que está acontecendo agora. Lugar: pipeline OTel + APM agêntico.

Camada 3 — Eval em produção. Subset do tráfego real é amostrado e avaliado (humano ou LLM-judge) com defasagem de minutos/horas. Detecta drift que não aparece em eval offline. Lugar: data warehouse + revisão periódica.

A pergunta diagnóstica: *"se um cliente reclama de um output de ontem, você consegue: (a) reproduzir o caso, (b) saber se foi padrão ou exceção, (c) testar o fix antes de subir?"*. Se a resposta para qualquer das três é "não", está faltando uma das três camadas.

3. Eval harness: o coração da disciplina

Eval harness é a infraestrutura que roda casos de teste, coleta outputs e produz métricas. Estrutura canônica:

```
eval_suite/
├─ cases/
│  ├─ happy/           # casos comuns, baseline de competência
│  ├─ edge/           # bordas do domínio, raros mas importantes
│  ├─ adversarial/     # tentativas de quebrar (prompt injection, etc)
│  └─ regression/     # casos que já falharam uma vez
├─ judges/
│  ├─ deterministic.py # regras formais
│  ├─ llm_judge.py     # avaliação por LLM
│  └─ human_inbox.py   # casos para revisão humana
├─ runners/
│  └─ parallel_runner.py
└─ reports/
    └─ 2026-06-08T14-22.html
```

Cinco propriedades canônicas de um eval harness sério:

1. **Casos são código versionado**, não Google Sheet. Cada caso tem `id`, `input`, `expected` (ou `oracle_uri`), `tags`.
2. **Determinismo onde possível**. Caso "soma 2+2" não precisa de LLM judge — bate exato ou falha.
3. **Custo medido por execução de suite**. Eval que custa US\$ 50 por PR rodada vira gargalo cultural.
4. **Outputs arquivados**. Mesmo após pass/fail, guarde o output. Permite investigar regressões de calibração ("antes acertava com 90% confiança, agora com 60%").
5. **Diff entre runs**. Pull request mostra cases que mudaram, não só pass count.

Sem eval harness, "melhoria do prompt" é exercício de pareidolia. Com ele, é engenharia.

4. Golden traces e a regressão semântica

Pareados aos casos de eval, mantenha **golden traces**: gravações completas de execuções consideradas perfeitas. Não só input/output — todo o caminho: decisões internas do agente, tools chamadas, prompts montados, raciocínio (se exposto).

```
{
  "trace_id": "golden-renegotiate-001",
  "input": "Cliente C-99 quer parcelar fatura INV-2026-0042",
  "expected_steps": [
    { "tool": "crm.read",      "args": { "customer_id": "C-99" } },
    { "tool": "billing.read", "args": { "invoice_id": "INV-2026-0042" } },
    { "decision": "policy_check",
      "rationale": "Cliente premium, fatura < 90 dias atraso → parcelar em até 6x" },
    { "tool": "billing.update",
      "args": { "invoice_id": "INV-2026-0042", "split": 6 } }
  ],
  "expected_output": {
    "status": "renegotiated",
    "installments": 6
  }
}
```

A **regressão semântica** detectada por golden traces é a regressão que escapa de tudo o mais: o output final está certo, mas o caminho mudou — o agente passou a consultar 4 tools em vez de 2, ou pulou o `policy_check`. O caso passa em assertion superficial e degrada em custo ou em compliance.

Política sã: ao detectar regressão semântica, falhar o PR com mensagem explícita ("trajetória mudou: +2 tool calls, -1 decisão"). Permitir override só com justificativa registrada no commit.

5. LLM-as-judge: poder, viés e calibração

Para outputs subjetivos (texto livre, classificações com nuance), o juiz tipicamente é outro LLM. Funciona — com calibração.

Vícios conhecidos de LLM-judges (documentados em pesquisa publicada):

- **Position bias.** Em comparação A/B, o LLM tende a preferir o primeiro (ou o último, dependendo do modelo).
- **Length bias.** Preferência por respostas mais longas, mesmo quando não são melhores.
- **Self-preference.** GPT-4 julgando outputs de GPT-4 dá nota maior. Claude julgando Claude, idem.
- **Sycophancy.** Se o prompt indica que o output veio de "um especialista", o juiz é mais leniente.

Mitigações canônicas:

```
def calibrated_judge(input, output_a, output_b):
    # 1. Randomizar ordem
    if random.random() > 0.5:
        a, b = output_b, output_a
        swapped = True
    else:
        a, b = output_a, output_b
        swapped = False

    # 2. Prompt blindado quanto à autoria
    score = llm_judge(
        prompt=BLIND_RUBRIC,
        criteria=["correctness", "concision", "actionability"],
        a=a, b=b
    )

    # 3. Rodar duas vezes; agregação
    score_2 = llm_judge(...)

    # 4. Calibrar contra golden human labels mensalmente
    if (score - score_2) > threshold:
        return "inconclusive_send_to_human"

    return invert_if(score, swapped)
```

A regra: **LLM-judge é estimador, não verdade.** Calibre periodicamente contra labels humanas em amostra; se a correlação cai abaixo de 0.7 para a tarefa, troque o judge ou pare de confiar nele.

6. Trace estruturado: o sucessor do log

Logs textuais (`logger.info("calling tool X")`) viraram passivo em sistemas agênticos. O substituto é **trace estruturado** seguindo padrão OpenTelemetry, com semantic conventions agênticas em maturação.

Spans canônicos em uma execução agêntica:

```
agent.task (root)
├─ agent.llm_call (model=claude-3.7)
│   ├── attr: prompt_tokens=2_847
│   ├── attr: completion_tokens=412
│   ├── attr: cost_usd=0.012
│   └─ attr: latency_ms=1_840
├─ agent.tool_call (mcp.crm.read_customer)
│   ├── attr: tool_input_hash=sha256:...
│   ├── attr: tool_output_size=1_204
│   └─ attr: latency_ms=85
├─ agent.delegation (to=billing-agent.acme)
│   ├── attr: protocol=a2a
│   ├── attr: task_id=t-9f3a
│   └─ agent.delegation_outcome (status=completed)
└─ agent.decision (kind=policy_check, outcome=allow)
    ├── attr: rule_id=premium_split_v3
    └─ attr: confidence=0.94
```

Convenções emergentes em 2026 (OTel GenAI semantic conventions):

- `gen_ai.system` — provedor do modelo.
- `gen_ai.request.model` — modelo exato com versão.
- `gen_ai.usage.input_tokens` / `output_tokens` .
- `gen_ai.response.finish_reason` .
- `agent.skill` — skill em execução.
- `agent.delegation.target` — DID do agente delegado.

O trace estruturado responde, sem código customizado, perguntas como: "qual tool consome mais tokens?", "qual skill tem maior latência p99?", "qual modelo tem mais falhas em policy check?". Sem trace estruturado, cada pergunta vira projeto de uma semana.

7. Métricas canônicas: latência, custo, qualidade, segurança

Quatro famílias de métricas que toda observability agêntica madura exporta:

Latência. - `task_duration_p50/p90/p99` — wall clock por task. - `llm_call_duration` por modelo e por skill. - `tool_call_duration` por tool. - `delegation_round_trip` para chamadas A2A/ACP.

Custo. - `cost_per_task` — agregado de tokens × preço. - `cost_per_skill` — útil para decidir preço (Volume VI). - `tokens_in / tokens_out` por modelo. - `cost_per_outcome` — custo dividido pelas tasks que terminaram em sucesso.

Qualidade. - `task_success_rate` por skill. - `human_override_rate` — quantas tasks foram corrigidas por humano. - `eval_score` em sliding window (eval em produção). - `regression_count` semana sobre semana.

Segurança. - `prompt_injection_attempts_detected`. - `policy_violation_rate`. - `escalation_to_human_rate`. - `unauthorized_tool_call_attempts`.

A regra: cada métrica precisa de **dono, alvo e ação**. Métrica sem alvo é decoração de dashboard. Métrica sem dono é métrica que ninguém olha.

8. SLOs agênticos: o que prometer ao consumidor da skill

Skill federada (Volume VI) sem SLO declarado vira pacote sem garantia. SLO agêntico canônico tem três compromissos quantificados:

```
slo:
  skill: "@acme/renegotiate-invoice@2.x"
  window: "30d rolling"
  targets:
    availability:
      indicator: "successful_invocations / total_invocations"
      target: 0.995
    latency:
      indicator: "p95(task_duration_seconds)"
      target: "< 8s"
    quality:
      indicator: "eval_score (LLM-judge calibrated, rolling)"
      target: "> 0.85"
  error_budget:
    monthly: "5h downtime + 0.5% quality degradation"
  remediation:
    quality_below_target:
      - rollback_to_previous_skill_version
      - escalate_to_skill_owner
```

A inovação face a SLOs tradicionais é o **target de qualidade**, que não é binário. Isso muda a operação: error budget agora consome por *degradação contínua*, não só por outage. Atingir 99.9% de uptime mas cair eval_score de 0.91 para 0.83 é incidente, mesmo sem 5xx.

9. Debugging distribuído: replay e bisseção

Quando um cliente reporta "ontem às 14:23 o agente respondeu X errado", o time precisa de duas capacidades:

Replay. Pegar o trace exato, rodar novamente, reproduzir o output. Requer: trace contendo input, modelo+versão, prompt completo, ferramentas chamadas (com responses gravadas), seed se aplicável. Sem replay, debug é especulação.

Bisseção temporal. "Quando esse caso começou a falhar?". Roda o caso contra deploys históricos da skill (binary search) até encontrar o deploy onde quebrou. Depende de versões da skill manterem-se executáveis e do harness aceitar `--at-version=2.2.7`.

```
$ eval bisect \  
  --case=ticket-99231 \  
  --good=v2.2.0 \  
  --bad=v2.3.1 \  
  --skill="@acme/renegotiate-invoice"  
  
Testing v2.2.5... PASS  
Testing v2.3.0... PASS  
Testing v2.3.1... FAIL  
→ Regressão introduzida em v2.3.1 (commit 7a3f9c2)  
→ Mudança suspeita: prompt do step "policy_check" alterado
```

Replay + bisseção transformam debug agêntico de "ler logs e adivinhar" para "isolar regressão em minutos". A diferença não é cosmética — é a diferença entre operar à noite e dormir em paz.

10. Manifesto: o que não é medido não está em produção

A frase "o que não é medido não pode ser melhorado" é truísmo gerencial clássico. Em sistemas agênticos ela tem corolário operacional duro:

O que não é medido **não está em produção** — está em fé.

Stack agêntica em produção sem eval harness, sem golden traces, sem trace estruturado e sem SLO de qualidade é deploy de protótipo disfarçado. Funciona até a primeira mudança silenciosa de modelo, até o primeiro caso edge no cliente errado, até o primeiro auditor perguntar "como vocês sabem que está funcionando?".

A resposta digna dessa pergunta é uma URL para dashboard que mostra: eval score na última semana, SLO compliance, regression count, custo por outcome. Resposta que começa com "deixa eu verificar com o time" é confissão de que a operação ainda é pre-industrial.

Tese final do volume: A maturidade de uma stack agêntica não se mede em quantos agentes ela tem ou em quão sofisticada é sua orquestração. Mede-se em **quanto tempo demora entre uma regressão acontecer e alguém saber**. Stacks profissionais medem isso em minutos. Stacks amadoras descobrem quando o cliente reclama.

Checklist Canônico — Observability Agêntica

- [] Eval harness com cases versionados em git rodando em CI.
- [] Golden traces capturados e em uso para detectar regressão semântica.
- [] LLM-judge calibrado mensalmente contra labels humanas.
- [] OTel GenAI semantic conventions adotadas em todos os spans.
- [] Métricas RED + custo + qualidade + segurança exportadas.
- [] SLO agêntico declarado por skill, incluindo target de qualidade.
- [] Replay funcional a partir do trace_id de qualquer execução.
- [] Bisseção temporal de skills implementada e usada em incidentes.
- [] Eval em produção amostra $\geq 1\%$ do tráfego e alimenta painel.
- [] Alerts disparam em degradação de eval_score, não só em 5xx.

Glossário do Volume

- **Eval harness** — infraestrutura que roda casos e produz métricas.
- **Golden trace** — gravação completa de execução considerada perfeita.
- **Regressão semântica** — output correto, mas trajetória degradada.
- **LLM-as-judge** — uso de LLM para avaliar outputs subjetivos.
- **OTel GenAI conventions** — atributos canônicos OpenTelemetry para IA.
- **SLO agêntico** — Service Level Objective incluindo target de qualidade.
- **Replay** — re-execução determinística a partir de trace gravado.
- **Bisseção** — busca binária para localizar versão que introduziu regressão.

Gancho para o Próximo Volume

Medir bem permite **ver** problemas. Mas a próxima fronteira é **prevenir** problemas: prompt injection cross-agent, ataques de side-channel via tools, cascatas de falha em rede de agentes. Esse é o terreno do **Volume IX — Segurança, Guardrails e Resiliência em Rede**.

